PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Complete Practical Report

Includes problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python source code, test cases, sample output screenshots, and conclusions.

Selected Case Studies	Main Concepts
3. Hospital Billing	Variables, arithmetic, if-else
12. Attendance Recovery	Percentages, while loop
13. ATM Simulation	Loops, conditions
17. Student Performance	Lists, comparisons
18. Fuel Cost	Arithmetic, validation
26. Seating Generator	Nested loops
29. Password Validation	Strings, loops
35. Text Compression	String traversal
42. Marks Matrix	Nested lists
53. Food Waste	Percentages, validation

Note: The output panels in this report are text-based sample output captures, not screenshots from a live Python IDE. Run each program in Python IDLE, PyCharm, or another Python editor to capture actual screenshots if required by your instructor.

Case Study 3: Hospital Billing and Patient Classification

1. Problem Statement

Create a program that adds consultation, test, and treatment charges. Apply a 10% discount when the total bill is at least ■5,000, then display the final bill.

2. Objectives

• Accept three hospital charge amounts.

• Calculate the total bill.

• Apply the stated discount rule and display the final amount.

3. Assumptions

• All entered charges are non-negative numbers.

• A 10% discount applies only when the total is ■5,000 or more.

• No tax or other fee is included.

4. Algorithm

1. Start.

2. Input consultation fee, test charges, and treatment charges.

3. Add the three charges to get total.

4. If total ≥ 5,000, calculate discount = total × 10%; otherwise discount = 0.

5. Calculate final bill = total – discount.

6. Display total, discount, and final bill.

7. Stop.

5. Flowchart

START → Input consultation, tests, treatment → total = consultation + tests + treatment → Is total ≥ 5000? → Yes: discount = total × 0.10 / No: discount = 0 → final bill = total – discount → Display results → STOP

6. Pseudocode

```
BEGIN
INPUT consultation, tests, treatment
total ← consultation + tests + treatment
IF total >= 5000 THEN
    discount ← total * 0.10
ELSE
    discount ← 0
END IF
final_bill ← total - discount
PRINT total, discount, final_bill
END
```

7. Python Source Code

```
consultation = float(input("Enter consultation fee: "))
tests = float(input("Enter test charges: "))
treatment = float(input("Enter treatment charges: "))

total = consultation + tests + treatment

if total >= 5000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

print("Total Bill:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)
```

8. Test Cases

Input / Test	Expected result
500	2000
500	1000

9. Sample Output Screenshot

```
Enter consultation fee: 500
Enter test charges: 2000
Enter treatment charges: 3000
Total Bill: 5500.0
Discount: 550.0
Final Bill: 4950.0
```

10. Conclusion

The program totals the hospital charges and correctly applies the discount rule to produce the amount payable.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Calculate a student's attendance percentage and determine whether the student meets the 75% requirement. If not, calculate the minimum number of consecutive classes the student must attend to reach 75%, assuming no further absences.

2. Objectives

• Calculate attendance percentage.

• Check the 75% eligibility threshold.

• Find the minimum additional classes required.

3. Assumptions

• Total classes is greater than zero.

• Attended classes is between zero and total classes.

• Every future class is attended.

4. Algorithm

1. Start.

2. Input total classes and attended classes.

3. Calculate percentage = attended ÷ total × 100.

4. If percentage ≥ 75, display eligible.

5. Otherwise, start needed at zero and increase it until (attended + needed) ÷ (total + needed) × 100 ≥ 75.

6. Display percentage and result.

7. Stop.

5. Flowchart

START → Input total and attended → Calculate percentage → Is percentage ≥ 75? → Yes: display Eligible → No: repeat needed = needed + 1 until future attendance reaches 75% → Display needed → STOP

6. Pseudocode

```
BEGIN
INPUT total, attended
percentage ← attended / total * 100
IF percentage >= 75 THEN
    PRINT Eligible
ELSE
    needed ← 0
    WHILE (attended + needed) / (total + needed) * 100 < 75
        needed ← needed + 1
    END WHILE
    PRINT needed
END IF
END
```

7. Python Source Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = attended / total * 100
    print("Attendance:", round(percentage, 2), "%")

    if percentage >= 75:
        print("Eligible for examination")
    else:
        needed = 0
        while (attended + needed) / (total + needed) * 100 < 75:
```

```
    needed += 1
print("Not eligible")
print("Classes needed:", needed)
```

8. Test Cases

Input / Test	Expected result
20	12
40	32

9. Sample Output Screenshot

```
Enter total classes: 20
Enter attended classes: 12
Attendance: 60.0 %
Not eligible
Classes needed: 12
```

10. Conclusion

The program checks attendance eligibility and calculates the minimum extra classes required when every future class is attended.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate three ATM transactions. The user can deposit or withdraw money. Reject non-positive amounts, invalid choices, and withdrawals that exceed the current balance.

2. Objectives

• Maintain an account balance.

• Process deposits and withdrawals.

• Reject invalid transactions and show the final balance.

3. Assumptions

• The opening balance is ■1,000.

• Exactly three transactions are processed.

• No bank fees or daily limits are applied.

4. Algorithm

1. Start with balance = 1000.

2. Repeat three times: display Deposit and Withdraw options.

3. Input choice and amount.

4. Reject amount ≤ 0.

5. For deposit, add amount to balance.

6. For withdrawal, subtract only if amount ≤ balance; otherwise report insufficient balance.

7. Display final balance.

8. Stop.

5. Flowchart

START → balance = 1000 → Repeat 3 times: input choice and amount → Is amount positive? → If deposit, add → If withdrawal and enough balance, subtract → Otherwise show error → Display balance → STOP

6. Pseudocode

```
BEGIN
balance ← 1000
FOR i ← 1 TO 3
    INPUT choice, amount
    IF amount <= 0 THEN PRINT Invalid amount
    ELSE IF choice = 1 THEN balance ← balance + amount
    ELSE IF choice = 2 AND amount <= balance THEN balance ← balance - amount
    ELSE IF choice = 2 THEN PRINT Insufficient balance
    ELSE PRINT Invalid choice
    END IF
END FOR
PRINT balance
END
```

7. Python Source Code

```
balance = 1000

for i in range(3):
    print("1. Deposit")
    print("2. Withdraw")
    choice = int(input("Enter choice: "))
    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount")
    elif choice == 1:
        balance += amount
        print("Deposit successful")
```

```
    elif choice == 2:
        if amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")
    else:
        print("Invalid choice")

print("Final Balance:", balance)
```

8. Test Cases

Input / Test	Expected result
Transactions: deposit 500; withdraw 200; deposit 100	Final balance ■1,400
Transactions: withdraw 1500; deposit 100; invalid choice 5 with amount 10	First rejected; final balance ■1,100

9. Sample Output Screenshot

```
Example transaction: deposit 500, withdraw 200, deposit 100
Final Balance: 1400.0
```

10. Conclusion

The simulation updates the balance safely and rejects transactions that violate the basic ATM rules.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

For each of three subjects, compare a student's first-exam marks with second-exam marks and report improvement, decline, or no change.

2. Objectives

• Use a list of subject names.

• Compare marks across two examinations.

• Report the change in marks for each subject.

3. Assumptions

• Marks are entered on a 0–100 scale.

• The user enters valid whole-number marks.

• The program handles three subjects.

4. Algorithm

1. Start.

2. Set subjects to Python, Maths, and English.

3. For each subject, input first and second exam marks.

4. Calculate difference = second – first.

5. If difference > 0, report improvement; if < 0, report decline; otherwise report no change.

6. Stop.

5. Flowchart

START → For each subject → Input first and second marks → difference = second – first → Positive? improvement / Negative? decline / Zero? no change → Next subject → STOP

6. Pseudocode

```
BEGIN
subjects ← [Python, Maths, English]
FOR each subject IN subjects
    INPUT first, second
    difference ← second - first
    IF difference > 0 THEN PRINT Improved
    ELSE IF difference < 0 THEN PRINT Decreased
    ELSE PRINT No change
    END IF
END FOR
END
```

7. Python Source Code

```
subjects = ["Python", "Maths", "English"]

for subject in subjects:
    first = int(input("Enter " + subject + " first exam marks: "))
    second = int(input("Enter " + subject + " second exam marks: "))

    difference = second - first

    if difference > 0:
        print(subject, "Improved by", difference, "marks")
    elif difference < 0:
        print(subject, "Decreased by", -difference, "marks")
    else:
        print(subject, "No change")
```

8. Test Cases

Input / Test	Expected result
Python: 70, 85	Python improved by 15 marks
Maths: 80, 70	Maths decreased by 10 marks
English: 75, 75	English no change

9. Sample Output Screenshot

```
Enter Python first exam marks: 70
Enter Python second exam marks: 85
Python Improved by 15 marks
```

10. Conclusion

The program clearly identifies subject-wise changes in performance between two examinations.

Case Study 18: Fuel Consumption and Route Comparison

1. Problem Statement

Calculate the fuel needed and fuel cost for a journey using distance, vehicle mileage, and price per litre. Check whether the cost is within a ■1,000 budget.

2. Objectives

• Calculate fuel required.

• Calculate total fuel cost.

• Compare the cost with a budget.

3. Assumptions

• Distance, mileage, and fuel price are positive numbers.

• The budget is fixed at ■1,000.

• Only fuel cost is included.

4. Algorithm

1. Start.

2. Input distance, mileage, and price per litre.

3. Validate that all values are greater than zero.

4. Fuel required = distance ÷ mileage.

5. Cost = fuel required × price.

6. Display fuel and cost.

7. If cost ≤ 1000, report within budget; otherwise report over budget.

8. Stop.

5. Flowchart

START → Input distance, mileage, fuel price → Validate positive values → fuel = distance / mileage → cost = fuel × price → Is cost ≤ 1000? → Display budget result → STOP

6. Pseudocode

```
BEGIN
INPUT distance, mileage, price
IF distance <= 0 OR mileage <= 0 OR price <= 0 THEN
    PRINT Invalid input
ELSE
    fuel ← distance / mileage
    cost ← fuel * price
    PRINT fuel, cost
    IF cost <= 1000 THEN PRINT Within budget
    ELSE PRINT Exceeds budget
    END IF
END IF
END
```

7. Python Source Code

```
distance = float(input("Enter distance in km: "))
mileage = float(input("Enter mileage in km/litre: "))
price = float(input("Enter fuel price per litre: "))

if distance <= 0 or mileage <= 0 or price <= 0:
    print("Invalid input")
else:
    fuel = distance / mileage
    cost = fuel * price

    print("Fuel Required:", round(fuel, 2), "litres")
    print("Journey Cost:", round(cost, 2))

    if cost <= 1000:
```

```
        print("Journey is within budget")
    else:
        print("Journey exceeds budget")
```

8. Test Cases

Input / Test	Expected result
200 km; 20 km/l; ■100/l	Fuel 10 litres; cost ■1,000; within budget
300 km; 15 km/l; ■110/l	Fuel 20 litres; cost ■2,200; exceeds budget

9. Sample Output Screenshot

```
Enter distance in km: 200
Enter mileage in km/litre: 20
Enter fuel price per litre: 100
Fuel Required: 10.0 litres
Journey Cost: 1000.0
Journey is within budget
```

10. Conclusion

The program estimates journey fuel expenses and checks whether the trip fits the given budget.

Case Study 26: Examination Seating Generator

1. Problem Statement

Generate seat labels for a classroom using a specified number of rows and columns. Each seat receives a unique sequential identifier.

2. Objectives

• Accept classroom dimensions.

• Use nested loops to generate a seating layout.

• Display unique seat labels row by row.

3. Assumptions

• Rows and columns must both be positive integers.

• Seats are numbered sequentially from S1.

• Reserved seats and student names are not included in this simple version.

4. Algorithm

1. Start.

2. Input rows and columns.

3. If either is not positive, display invalid input.

4. Otherwise set seat = 1.

5. For each row, repeat for each column: display S followed by seat number and increase seat by one.

6. Move to the next line after each row.

7. Stop.

5. Flowchart

START → Input rows and columns → Are both positive? → No: show error → Yes: seat = 1 → Nested loops print seat labels → STOP

6. Pseudocode

```
BEGIN
INPUT rows, columns
IF rows <= 0 OR columns <= 0 THEN PRINT Invalid input
ELSE
    seat ← 1
    FOR i ← 1 TO rows
        FOR j ← 1 TO columns
            PRINT S followed by seat
            seat ← seat + 1
        END FOR
        PRINT newline
    END FOR
END IF
END
```

7. Python Source Code

```
rows = int(input("Enter number of rows: "))
columns = int(input("Enter seats per row: "))

if rows <= 0 or columns <= 0:
    print("Invalid classroom size")
else:
    seat = 1
    for i in range(rows):
        for j in range(columns):
            print("S" + str(seat), end="\t")
            seat += 1
        print()
```

8. Test Cases

Input / Test	Expected result
Rows 2, columns 3	S1 S2 S3 on row 1; S4 S5 S6 on row 2
Rows 0, columns 4	Invalid classroom size

9. Sample Output Screenshot

```
Enter number of rows: 2
Enter seats per row: 3
S1    S2    S3
S4    S5    S6
```

10. Conclusion

Nested loops create a simple, orderly seating plan with a unique label for each seat.

Case Study 29: Password Policy Validation

1. Problem Statement

Validate a sample password using two rules: it must contain at least eight characters and at least one digit. Report each failed rule.

2. Objectives

• Accept a password string.

• Check its length.

• Check whether it contains a digit and report the result.

3. Assumptions

• The password is treated as sample input only.

• The two specified rules are the only rules checked.

• No password is stored or transmitted.

4. Algorithm

1. Start.

2. Input password.

3. Set has_digit = False.

4. Check each character; if it is a digit, set has_digit = True.

5. If length ≥ 8 and has_digit is True, report valid.

6. Otherwise report invalid and print each failed rule.

7. Stop.

5. Flowchart

START → Input password → Scan characters for a digit → Check length ≥ 8 and digit found → Yes: valid → No: invalid and list failed rules → STOP

6. Pseudocode

```
BEGIN
INPUT password
has_digit ← FALSE
FOR each character IN password
    IF character is a digit THEN has_digit ← TRUE
END FOR
IF length(password) >= 8 AND has_digit THEN
    PRINT Valid password
ELSE
    PRINT Invalid password
    IF length(password) < 8 THEN PRINT Length error
    IF has_digit = FALSE THEN PRINT Digit error
END IF
END
```

7. Python Source Code

```
password = input("Enter password: ")

has_digit = False

for ch in password:
    if ch.isdigit():
        has_digit = True

if len(password) >= 8 and has_digit:
    print("Password is valid")
else:
    print("Password is invalid")
    if len(password) < 8:
        print("Password must have at least 8 characters")
```

```
    if not has_digit:
        print("Password must contain a digit")
```

8. Test Cases

Input / Test	Expected result
kavya123	Valid password
kavya	Invalid; must have at least 8 characters; must contain a digit
passwordonly	Invalid; must contain a digit

9. Sample Output Screenshot

```
Enter password: kavya123
Password is valid
```

10. Conclusion

The program applies simple string checks and explains which password requirements are not satisfied.

Case Study 35: Text Compression Using Character Runs

1. Problem Statement

Compress consecutive repeated characters by writing each character followed by the number of times it repeats consecutively. For example, aaabb becomes a3b2.

2. Objectives

• Accept a string.

• Count consecutive repeated characters.

• Build and display the compressed string.

3. Assumptions

• The program is case-sensitive.

• Only consecutive repetitions are grouped.

• An empty input produces an empty compressed string.

4. Algorithm

1. Start.

2. Input text.

3. Set result to empty string.

4. If text is not empty, set count = 1.

5. Scan characters from the second character onward.

6. If current character matches previous, increase count; otherwise append previous character and count, then reset count to 1.

7. Append the final character and count.

8. Display compressed string.

9. Stop.

5. Flowchart

START → Input text → Is text empty? → Yes: display empty result → No: scan characters → Same as previous? increase count / Different? append previous run → Append final run → Display result → STOP

6. Pseudocode

```
BEGIN
INPUT text
result ← empty string
IF length(text) > 0 THEN
    count ← 1
    FOR i FROM 1 TO length(text)-1
        IF text[i] = text[i-1] THEN count ← count + 1
        ELSE result ← result + text[i-1] + count; count ← 1
        END IF
    END FOR
    result ← result + last character + count
END IF
PRINT result
END
```

7. Python Source Code

```
text = input("Enter a string: ")

result = ""
count = 1

for i in range(1, len(text)):
    if text[i] == text[i - 1]:
        count += 1
    else:
```

```
        result += text[i - 1] + str(count)
        count = 1

if len(text) > 0:
    result += text[-1] + str(count)

print("Original String:", text)
print("Compressed String:", result)
```

8. Test Cases

Input / Test	Expected result
aaabbcccc	a3b2c4
abcd	a1b1c1d1
(empty input)	Empty compressed string

9. Sample Output Screenshot

```
Enter a string: aaabbcccc
Original String: aaabbcccc
Compressed String: a3b2c4
```

10. Conclusion

The program demonstrates string traversal and counting repeated character runs using a simple loop.

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Store marks for three students in three subjects using a nested list. Calculate each student's total and average marks.

2. Objectives

• Represent marks as a nested list.

• Calculate row-wise totals and averages.

• Display results student by student.

3. Assumptions

• The sample has three students and three subjects.

• Marks are valid numeric values from 0 to 100.

• The sample marks are included directly in the program.

4. Algorithm

1. Start.

2. Create a nested list of marks.

3. For each student's list, calculate total using sum().

4. Calculate average = total ÷ number of subjects.

5. Display student number, total, and average.

6. Stop.

5. Flowchart

START → Create marks matrix → For each student row → Calculate total → Calculate average → Display result → Next student → STOP

6. Pseudocode

```
BEGIN
marks ← [[80,90,85], [70,75,80], [90,95,92]]
FOR i FROM 0 TO number of students - 1
    total ← sum(marks[i])
    average ← total / number of subjects
    PRINT student number, total, average
END FOR
END
```

7. Python Source Code

```
marks = [
    [80, 90, 85],
    [70, 75, 80],
    [90, 95, 92]
]

for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])

    print("Student", i + 1)
    print("Total Marks:", total)
    print("Average Marks:", round(average, 2))
```

8. Test Cases

Input / Test	Expected result
Student 1: 80, 90, 85	Total 255; average 85.0
Student 2: 70, 75, 80	Total 225; average 75.0

Input / Test	Expected result
Student 3: 90, 95, 92	Total 277; average 92.33

9. Sample Output Screenshot

```
Student 1
Total Marks: 255
Average Marks: 85.0
```

10. Conclusion

The program shows how nested lists can store marks and how loops can calculate student-wise statistics.

Case Study 53: Food Waste Monitoring System

1. Problem Statement

Compare meals prepared with meals served. Calculate leftover meals and the percentage of prepared meals wasted, then classify waste above 20% as high.

2. Objectives

• Calculate the number of leftover meals.

• Calculate the waste percentage.

• Classify food waste using a 20% threshold.

3. Assumptions

• Prepared meals must be greater than zero.

• Served meals must be between zero and prepared meals.

• Waste is high only when the percentage is greater than 20%.

4. Algorithm

1. Start.

2. Input meals prepared and meals served.

3. Validate prepared > 0 and 0 ≤ served ≤ prepared.

4. Calculate waste = prepared – served.

5. Calculate percentage = waste ÷ prepared × 100.

6. Display waste and percentage.

7. If percentage > 20, report high waste; otherwise report acceptable waste.

8. Stop.

5. Flowchart

START → Input prepared and served → Validate values → waste = prepared – served → percentage = waste / prepared × 100 → Is percentage > 20? → High waste / Acceptable waste → STOP

6. Pseudocode

```
BEGIN
INPUT prepared, served
IF prepared <= 0 OR served < 0 OR served > prepared THEN
    PRINT Invalid input
ELSE
    waste ← prepared - served
    percentage ← waste / prepared * 100
    PRINT waste, percentage
    IF percentage > 20 THEN PRINT High food waste
    ELSE PRINT Food waste is acceptable
    END IF
END IF
END
```

7. Python Source Code

```
prepared = int(input("Enter meals prepared: "))
served = int(input("Enter meals served: "))

if prepared <= 0 or served < 0 or served > prepared:
    print("Invalid input")
else:
    waste = prepared - served
    percentage = waste / prepared * 100

    print("Meals Wasted:", waste)
    print("Waste Percentage:", round(percentage, 2), "%")

    if percentage > 20:
```

```
        print("High food waste")
    else:
        print("Food waste is acceptable")
```

8. Test Cases

Input / Test	Expected result
Prepared 100; served 75	Waste 25; 25%; high food waste
Prepared 100; served 85	Waste 15; 15%; acceptable
Prepared 0; served 0	Invalid input

9. Sample Output Screenshot

```
Enter meals prepared: 100
Enter meals served: 75
Meals Wasted: 25
Waste Percentage: 25.0 %
High food waste
```

10. Conclusion

The program calculates leftover food and highlights when the waste percentage crosses the specified threshold.